

Reasoning in Linear Temporal Logics on Finite Traces

Game-theoretic Approach to LTLf Planning and Synthesis

Giuseppe De Giacomo



Key point

LTL_f formulas can be translated into a finite-state automaton on finite words $\mathcal{A}_\varphi$ such that:

$$t \models \varphi \text{ iff } t \in \mathcal{L}(\mathcal{A}_\varphi)$$

- in **linear time** if $\mathcal{A}_\varphi$ is an **Alternating Finite-state Automata** (AFA);
- in **exponential time** if $\mathcal{A}_\varphi$ is an **Nondeterministic Finite-state Automaton** (NFA);
- in **double exponential time** if $\mathcal{A}_\varphi$ is an **Deterministic Finite-state Automaton** (DFA).

We can compile reasoning into automata based procedures!

Key point

LTL_f formulas can be translated into a finite-state automaton on finite words $\mathcal{A}_\varphi$ such that:

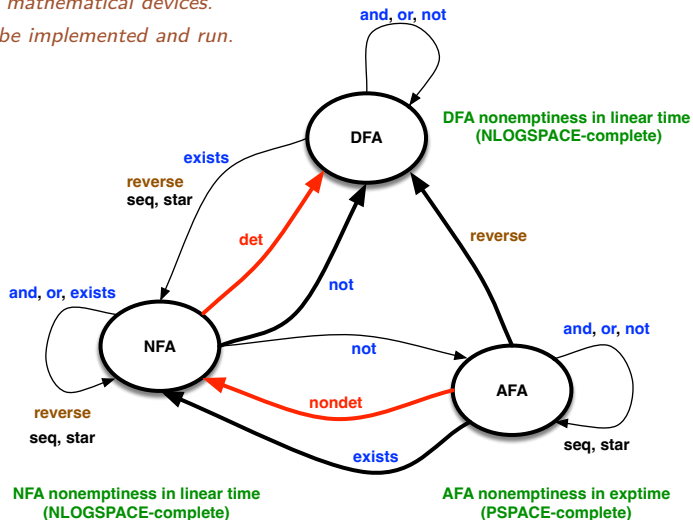
$$t \models \varphi \text{ iff } t \in \mathcal{L}(\mathcal{A}_\varphi)$$

- in **linear time** if $\mathcal{A}_\varphi$ is an **Alternating Finite-state Automata** (AFA);
- in **exponential time** if $\mathcal{A}_\varphi$ is an **Nondeterministic Finite-state Automaton** (NFA);
- in **double exponential time** if $\mathcal{A}_\varphi$ is an **Deterministic Finite-state Automaton** (DFA).

We can compile reasoning into automata based procedures!

Summary of automata theory on finite sequences:

- NFA's and AFA's are mathematical devices.
- DFA's, instead, can be implemented and run.



Key point

LTL_f formulas can be translated into a finite-state automaton on finite words $\mathcal{A}_\varphi$ such that:

$$t \models \varphi \text{ iff } t \in \mathcal{L}(\mathcal{A}_\varphi)$$

- in **linear time** if $\mathcal{A}_\varphi$ is an **Alternating Finite-state Automata** (AFA);
- in exponential time if $\mathcal{A}_\varphi$ is an Nondeterministic Finite-state Automaton (NFA);
- in double exponential time if $\mathcal{A}_\varphi$ is an Deterministic Finite-state Automaton (DFA).

We can compile reasoning into automata based procedures!

Alternating Automata on Finite Words (AFA)

$$\mathcal{A} = (2^{\mathcal{P}}, Q, q_0, \delta, F)$$

- $2^{\mathcal{P}}$ alphabet
- Q is a finite nonempty set of states
- q_0 is the initial state
- F is a set of accepting states
- δ is a **transition function** $\delta : Q \times 2^{\mathcal{P}} \rightarrow B^+(Q)$, where $B^+(Q)$ is a set of positive boolean formulas whose atoms are states of Q .

AFA run

Given an **input word** $a_0, a_1, \dots, a_{n-1}$, an **AFA run** of an AFA is a **tree** (rather than a sequence) labelled by states of AFA such that

- root is labelled by q_0 ;
- if node x at level i is labelled by a state q and $\delta(q, a_i) = \Theta$, then either Θ is **true** or some $P \subseteq Q$ satisfies Θ and x has a child for each element in P .

A run is **accepting** if all leaves at depth n are labeled by states in F . Thus, a branch in an accepting run has to hit the **true** transition or hit an **accepting state** after reading all the input word $a_0, a_1, \dots, a_{n-1}$.

(We adopt notation of “An Automata-Theoretic Approach to Linear Temporal Logic” by Moshe Vardi, 1996).

AFA $\mathcal{A}_\varphi$ associated with an LTL_f formula φ (in NNF)

$\mathcal{A}_\varphi = (2^{\mathcal{P}}, CL_\varphi, "\varphi", \delta, F)$ where

- $2^{\mathcal{P}}$ is the alphabet ($\mathcal{P}$ includes a special proposition *Last* to denote the last element of the trace),
- CL_φ is the state set
- $"\varphi"$ is the initial state
- $F = \emptyset$ is the set of final states, which is empty
- δ is the transition function, defined as:

$$\begin{aligned}
 \delta("A", \Pi) &= \text{true if } A \in \Pi \\
 \delta("A", \Pi) &= \text{false if } A \notin \Pi \\
 \delta(" \neg A", \Pi) &= \text{false if } A \in \Pi \\
 \delta(" \neg A", \Pi) &= \text{true if } A \notin \Pi \\
 \delta("\varphi_1 \wedge \varphi_2", \Pi) &= \delta("\varphi_1", \Pi) \wedge \delta("\varphi_2", \Pi) \\
 \delta("\varphi_1 \vee \varphi_2", \Pi) &= \delta("\varphi_1", \Pi) \vee \delta("\varphi_2", \Pi) \\
 \delta(" \bigcirc \varphi", \Pi) &= \begin{cases} "\varphi" & \text{if } Last \notin \Pi \\ \text{false} & \text{if } Last \in \Pi \end{cases} \\
 \delta(" \bigcirc \varphi", \Pi) &= \delta("\varphi", \Pi) \vee \delta(" \bigcirc \bigcirc \varphi", \Pi) \\
 \delta("\varphi_1 \mathcal{U} \varphi_2", \Pi) &= \delta("\varphi_2", \Pi) \vee (\delta("\varphi_1", \Pi) \wedge \delta(" \bigcirc (\varphi_1 \mathcal{U} \varphi_2)", \Pi)) \\
 \delta(" \bullet \varphi", \Pi) &= \begin{cases} "\varphi" & \text{if } Last \notin \Pi \\ \text{true} & \text{if } Last \in \Pi \end{cases} \\
 \delta(" \square \varphi", \Pi) &= \delta("\varphi", \Pi) \wedge \delta(" \bullet \square \varphi", \Pi) \\
 \delta("\varphi_1 \mathcal{R} \varphi_2", \Pi) &= \delta("\varphi_2", \Pi) \wedge (\delta("\varphi_1", \Pi) \vee \delta(" \bullet (\varphi_1 \mathcal{R} \varphi_2)", \Pi))
 \end{aligned}$$

We put the LTL_f formula in NNF, because AFA's transitions return positive boolean combinations of states ($B^+(Q)$).

NNF

Negation Normal Form for LTL_f : for $a \in AP$

$$\varphi ::= \text{true} \mid \text{false} \mid A \mid \neg A \mid \varphi \wedge \varphi \mid \varphi \vee \varphi \mid \bigcirc \varphi \mid \bullet \varphi \mid \Diamond \varphi \mid \Box \varphi \mid \varphi \mathcal{U} \varphi \mid \varphi \mathcal{R} \varphi$$

Each LTL_f formula φ admits an equivalent in NNF denoted $nnf(\varphi)$, which is obtained in linear time in the size formula by **pushing negation all the way**, exploiting duals through the follow equivalence:

- $\neg \neg \varphi \equiv \varphi$
- $\neg(\varphi_1 \wedge \varphi_2) \equiv \neg \varphi_1 \vee \neg \varphi_2$
- $\neg(\varphi_1 \vee \varphi_2) \equiv \neg \varphi_1 \wedge \neg \varphi_2$
- $\neg \bigcirc \varphi \equiv \bullet \neg \varphi$
- $\neg \bullet \varphi \equiv \bigcirc \neg \varphi$
- $\neg \Diamond \varphi \equiv \Box \neg \varphi$
- $\neg \Box \varphi \equiv \Diamond \neg \varphi$
- $\neg(\varphi_1 \mathcal{U} \varphi_2) \equiv \neg \varphi_1 \mathcal{R} \neg \varphi_2$
- $\neg(\varphi_1 \mathcal{R} \varphi_2) \equiv \neg \varphi_1 \mathcal{U} \neg \varphi_2$

States of the AFA $\mathcal{A}_\varphi$

The states of $\mathcal{A}_\varphi$ are the subformulas of φ once expanded using the fixpoint equivalence.
This set of formulas is called the **syntactic closure** of φ .

Syntactic Closure of an LTL_f formula

The syntactic closure, aka “Fisher-Ladner closure”, CL_φ of an LTL_f formula φ is a set of LTL_f formulas inductively defined as follows:

- $\varphi \in CL_\varphi$
- $\neg A \in CL_\varphi$ if $A \in CL_\varphi$
- $A \in CL_\varphi$ if $\neg A \in CL_\varphi$
- $\varphi_1 \wedge \varphi_2 \in CL_\varphi$ implies $\varphi_1, \varphi_2 \in CL_\varphi$
- $\varphi_1 \vee \varphi_2 \in CL_\varphi$ implies $\varphi_1, \varphi_2 \in CL_\varphi$
- $\bigcirc \varphi \in CL_\varphi$ implies $\varphi \in CL_\varphi$
- $\Diamond \varphi \in CL_\varphi$ implies $\varphi, \bigcirc \Diamond \varphi \in CL_\varphi$
- $\varphi_1 \mathcal{U} \varphi_2 \in CL_\varphi$ implies $\varphi_1, \varphi_2, \bigcirc(\varphi_1 \mathcal{U} \varphi_2) \in CL_\varphi$
- $\bullet \varphi \in CL_\varphi$ implies $\varphi \in CL_\varphi$
- $\Box \varphi \in CL_\varphi$ implies $\varphi, \bullet \Box \varphi \in CL_\varphi$
- $\varphi_1 \mathcal{R} \varphi_2 \in CL_\varphi$ implies $\varphi_1, \varphi_2, \bullet(\varphi_1 \mathcal{R} \varphi_2) \in CL_\varphi$

Observe that the cardinality of CL_φ is **linear in the size of φ** .

LTL_f fixpoint equations

- $\Diamond \varphi \equiv \varphi \vee \bigcirc(\Diamond \varphi)$
- $\Box \varphi \equiv \varphi \wedge \bullet(\Box \varphi)$
- $\varphi_1 \mathcal{U} \varphi_2 \equiv \varphi_2 \vee (\varphi_1 \wedge \bigcirc(\varphi_1 \mathcal{U} \varphi_2))$
- $\varphi_1 \mathcal{R} \varphi_2 \equiv \varphi_2 \wedge (\varphi_1 \vee \bullet(\varphi_1 \mathcal{R} \varphi_2))$

LTL_f and Automata

AFA $\mathcal{A}_\varphi$ associated with an LTL_f formula φ (in NNF)

$\mathcal{A}_\varphi = (2^{\mathcal{P}}, CL_\varphi, " \varphi ", \delta, F)$ where

- $2^{\mathcal{P}}$ is the alphabet,
- CL_φ is the state set,
- $" \varphi "$ is the initial state
- $F = \emptyset$ is the empty set of final states
- δ is the transition function

Transition function δ

$$\begin{aligned}
 \delta("A", \Pi) &= \text{true if } A \in \Pi \\
 \delta("A", \Pi) &= \text{false if } A \notin \Pi \\
 \delta(" \neg A ", \Pi) &= \text{false if } A \in \Pi \\
 \delta(" \neg A ", \Pi) &= \text{true if } A \notin \Pi \\
 \delta(" \varphi_1 \wedge \varphi_2 ", \Pi) &= \delta(" \varphi_1 ", \Pi) \wedge \delta(" \varphi_2 ", \Pi) \\
 \delta(" \varphi_1 \vee \varphi_2 ", \Pi) &= \delta(" \varphi_1 ", \Pi) \vee \delta(" \varphi_2 ", \Pi) \\
 \delta(" \bigcirc \varphi ", \Pi) &= \begin{cases} " \varphi " & \text{if } Last \notin \Pi \\ \text{false} & \text{if } Last \in \Pi \end{cases} \\
 \delta(" \Diamond \varphi ", \Pi) &= \delta(" \varphi ", \Pi) \vee \delta(" \bigcirc \Diamond \varphi ", \Pi) \\
 \delta(" \varphi_1 \mathcal{U} \varphi_2 ", \Pi) &= \delta(" \varphi_2 ", \Pi) \vee (\delta(" \varphi_1 ", \Pi) \wedge \delta(" \bigcirc (\varphi_1 \mathcal{U} \varphi_2) ", \Pi)) \\
 \delta(" \bullet \varphi ", \Pi) &= \begin{cases} " \varphi " & \text{if } Last \notin \Pi \\ \text{true} & \text{if } Last \in \Pi \end{cases} \\
 \delta(" \Box \varphi ", \Pi) &= \delta(" \varphi ", \Pi) \wedge \delta(" \bullet \Box \varphi ", \Pi) \\
 \delta(" \varphi_1 \mathcal{R} \varphi_2 ", \Pi) &= \delta(" \varphi_2 ", \Pi) \wedge (\delta(" \varphi_1 ", \Pi) \vee \delta(" \bullet (\varphi_1 \mathcal{R} \varphi_2) ", \Pi))
 \end{aligned}$$

LTL_f fixpoint equations

- $\Diamond \varphi \equiv \varphi \vee \bigcirc (\Diamond \varphi)$
- $\Box \varphi \equiv \varphi \wedge \bullet (\Box \varphi)$
- $\varphi_1 \mathcal{U} \varphi_2 \equiv \varphi_2 \vee (\varphi_1 \wedge \bigcirc (\varphi_1 \mathcal{U} \varphi_2))$
- $\varphi_1 \mathcal{R} \varphi_2 \equiv \varphi_2 \wedge (\varphi_1 \vee \bullet (\varphi_1 \mathcal{R} \varphi_2))$

LTL_f and Automata

AFAs can be transformed into NFA with standard algorithms in **exponential time**.

NFA A_φ associated with an LTL_f formula φ (in NNF)

δ transition function

$$\begin{aligned}
 \delta("A", \Pi) &= \text{true if } A \in \Pi \\
 \delta("A", \Pi) &= \text{false if } A \notin \Pi \\
 \delta(" \neg A", \Pi) &= \text{false if } A \in \Pi \\
 \delta(" \neg A", \Pi) &= \text{true if } A \notin \Pi \\
 \delta(" \varphi_1 \wedge \varphi_2", \Pi) &= \delta(" \varphi_1", \Pi) \wedge \delta(" \varphi_2", \Pi) \\
 \delta(" \varphi_1 \vee \varphi_2", \Pi) &= \delta(" \varphi_1", \Pi) \vee \delta(" \varphi_2", \Pi) \\
 \delta(" \bigcirc \varphi", \Pi) &= \begin{cases} " \varphi" & \text{if } \text{Last} \notin \Pi \\ \text{false} & \text{if } \text{Last} \in \Pi \end{cases} \\
 \delta(" \Diamond \varphi", \Pi) &= \delta(" \varphi", \Pi) \vee \delta(" \bigcirc \Diamond \varphi", \Pi) \\
 \delta(" \varphi_1 \mathcal{U} \varphi_2", \Pi) &= \delta(" \varphi_2", \Pi) \vee (\delta(" \varphi_1", \Pi) \wedge \delta(" \bigcirc (\varphi_1 \mathcal{U} \varphi_2)", \Pi)) \\
 \delta(" \bullet \varphi", \Pi) &= \begin{cases} " \varphi" & \text{if } \text{Last} \notin \Pi \\ \text{true} & \text{if } \text{Last} \in \Pi \end{cases} \\
 \delta(" \Box \varphi", \Pi) &= \delta(" \varphi", \Pi) \wedge \delta(" \bullet \Box \varphi", \Pi) \\
 \delta(" \varphi_1 \mathcal{R} \varphi_2", \Pi) &= \delta(" \varphi_2", \Pi) \wedge (\delta(" \varphi_1", \Pi) \vee \delta(" \bullet (\varphi_1 \mathcal{R} \varphi_2)", \Pi))
 \end{aligned}$$

AFA2NFA transformation

algorithm LTL_f2NFA

input LTL_f formula φ

output NFA $A_\varphi = (2^{\mathcal{P}}, S, \{s_0\}, q, \{s_f\})$

$s_0 \leftarrow \{" \varphi "\}$

▷ single initial state

$s_f \leftarrow \emptyset$

▷ single final state

$S \leftarrow \{s_0, s_f\}, q \leftarrow \emptyset$

while (S or q change) **do**

if ($q \in S$ and $q' \models \bigwedge_{(" \psi" \in q)} \delta(" \psi", \Pi)$)

$S \leftarrow S \cup \{q'\}$

▷ update set of states

$q \leftarrow q \cup \{(q, \Pi, q')\}$

▷ update transition relation

end while

Using function δ we can build the NFA A_φ of an LTL_f formula φ in a forward fashion. States of A_φ are sets of atoms (recall that each atom is quoted φ subformulas) to be interpreted as a conjunction; the empty conjunction $\emptyset$ stands for **true**.

LTL_f and Automata

LTL_f formulas can be directly translated in **exponential time** to NFAs, using AFA only implicitly.

NFA $\mathcal{A}_\varphi$ associated with an LTL_f formula φ (in NNF)

Auxiliary rules

$\delta("A", \Pi)$	= true if $A \in \Pi$
$\delta("A", \Pi)$	= false if $A \notin \Pi$
$\delta("¬A", \Pi)$	= false if $A \in \Pi$
$\delta("¬A", \Pi)$	= true if $A \notin \Pi$
$\delta("\varphi_1 \wedge \varphi_2", \Pi)$	= $\delta("\varphi_1", \Pi) \wedge \delta("\varphi_2", \Pi)$
$\delta("\varphi_1 \vee \varphi_2", \Pi)$	= $\delta("\varphi_1", \Pi) \vee \delta("\varphi_2", \Pi)$
$\delta("\bigcirc\varphi", \Pi)$	= $\begin{cases} "\varphi" & \text{if } Last \notin \Pi \\ \text{false} & \text{if } Last \in \Pi \end{cases}$
$\delta("\Diamond\varphi", \Pi)$	= $\delta("\varphi", \Pi) \vee \delta("\bigcirc\Diamond\varphi", \Pi)$
$\delta("\varphi_1 \mathcal{U} \varphi_2", \Pi)$	= $\delta("\varphi_2", \Pi) \vee (\delta("\varphi_1", \Pi) \wedge \delta("\bigcirc(\varphi_1 \mathcal{U} \varphi_2)", \Pi))$
$\delta("\bullet\varphi", \Pi)$	= $\begin{cases} "\varphi" & \text{if } Last \notin \Pi \\ \text{true} & \text{if } Last \in \Pi \end{cases}$
$\delta("\Box\varphi", \Pi)$	= $\delta("\varphi", \Pi) \wedge \delta("\bullet\Box\varphi", \Pi)$
$\delta("\varphi_1 \mathcal{R} \varphi_2", \Pi)$	= $\delta("\varphi_2", \Pi) \wedge (\delta("\varphi_1", \Pi) \vee \delta("\bullet(\varphi_1 \mathcal{R} \varphi_2)", \Pi))$

Observe these are the rules defining the transition function of the AFA!

Using function δ we can build the NFA $\mathcal{A}_\varphi$ of an LTL_f formula φ in a forward fashion. States of $\mathcal{A}_\varphi$ are sets of atoms (recall that each atom is quoted φ subformulas) to be interpreted as a conjunction; the empty conjunction $\emptyset$ stands for **true**.

Algorithm: LTL_f2NFA

algorithm LTL_f2NFA

input LTL_f formula φ

output NFA $\mathcal{A}_\varphi = (2^{\mathcal{P}}, \mathcal{S}, \{s_0\}, q, \{s_f\})$

$s_0 \leftarrow \{"\varphi"\}$

▷ single initial state

$s_f \leftarrow \emptyset$

▷ single final state

$\mathcal{S} \leftarrow \{s_0, s_f\}, q \leftarrow \emptyset$

while ($\mathcal{S}$ or q change) **do**

if ($q \in \mathcal{S}$ and $q' \models \bigwedge_{("\psi" \in q)} \delta("\psi", \Pi)$)

$\mathcal{S} \leftarrow \mathcal{S} \cup \{q'\}$

▷ update set of states

$q \leftarrow q \cup \{(q, \Pi, q')\}$

▷ update transition relation

end while

In building the AFA and then the NFA we assume to have a special proposition *Last* $\in \mathcal{P}$.

Removing the special proposition *Last*

If we want to remove such an assumption, we can easily transform the obtained NFA

$$A_\varphi = (2^{\mathcal{P}}, S, \{ \varphi \}, \varrho, \{ \emptyset \}) \quad \text{into the new NFA} \quad A'_\varphi = (2^{\mathcal{P}'}, S', S_0, \varrho', F')$$

where:

- $\mathcal{P}' = \mathcal{P} - \{ \textit{Last} \};$
- $S'_0 = \{ s_0 \};$
- $S' = S \cup \{ \textit{ended} \};$
- $F' = \{ \emptyset, \textit{ended} \};$
- $(q, \Pi', q') \in \varrho'$ iff $\begin{cases} (q, \Pi', q') \in \varrho \text{ or} \\ (q, \Pi' \cup \{ \textit{Last} \}, \emptyset) \in \varrho \text{ and } q' = \textit{ended} \end{cases}$

Example (NFA for $\Box A$)

The NFA for $\Box A$ is as follows:

- Initial state $\{\Box A\}$;
- Final state $\{\emptyset\}$;
- Transitions:
 - ▶ $\rho_n(\{\Box A\}, A \wedge Last, q')$ with $q' \models \delta(\Box A, A \wedge Last) = \delta(A, A \wedge Last) \wedge \delta(\bullet \Box A, A \wedge Last) = \text{true} \wedge \delta(\bullet \Box A, A \wedge Last)$, i.e., $q' = \{\emptyset\}$;
 - ▶ $\rho_n(\{\Box A\}, A \wedge \neg Last, q')$ with $q' \models \delta(\Box A, A \wedge \neg Last) = \delta(A, A \wedge \neg Last) \wedge \delta(\bullet \Box A, A \wedge \neg Last) = \text{true} \wedge \delta(\bullet \Box A, A \wedge \neg Last)$, i.e., $q' = \{\Box A\}$;
 - ▶ $\rho_n(\{\Box A\}, \neg A, q')$ with $q' \models \delta(\Box A, \neg A) = \delta(A, \neg A) \wedge \delta(\bullet \Box A, \neg A) = \text{false} \wedge \delta(\bullet \Box A, \neg A)$, i.e., there are not such q' . (Notice same behavior with $Last$ and $\neg Last$.)

Example (NFA for $\Diamond A$)

The NFA for $\Diamond A$ is as follows:

- Initial state $\{\Diamond A\}$;
- Final state $\{\emptyset\}$;
- Transitions:
 - ▶ $\rho_n(\{\Diamond A\}, A \wedge Last, q')$ with $q' \models \delta(\Diamond A, A \wedge Last) = \delta(A, A \wedge Last) \vee \delta(\bigcirc \Diamond A, A \wedge Last) = \text{true} \vee \text{false}$, i.e., $q' = \{\emptyset\}$;
 - ▶ $\rho_n(\{\Diamond A\}, A \wedge \neg Last, q')$ with $q' \models \delta(\Diamond A, A \wedge \neg Last) = \delta(A, A \wedge \neg Last) \vee \delta(\bigcirc \Diamond A, A \wedge \neg Last) = \text{true} \vee \Diamond A$, i.e., $q' = \{\emptyset\}$;
 - ▶ $\rho_n(\{\Diamond A\}, \neg A \wedge Last, q')$ with $q' \models \delta(\Diamond A, \neg A \wedge Last) = \delta(A, \neg A \wedge Last) \vee \delta(\bigcirc \Diamond A, \neg A \wedge Last) = \text{false} \vee \text{false}$, i.e., no such q' exists;
 - ▶ $\rho_n(\{\Diamond A\}, \neg A \wedge \neg Last, q')$ with $q' \models \delta(\Diamond A, \neg A \wedge \neg Last) = \delta(A, \neg A \wedge \neg Last) \vee \delta(\bigcirc \Diamond A, \neg A \wedge \neg Last) = \text{false} \vee \delta(\bigcirc \Diamond A, \neg A \wedge \neg Last)$, i.e., $q' = \{\Diamond A\}$.

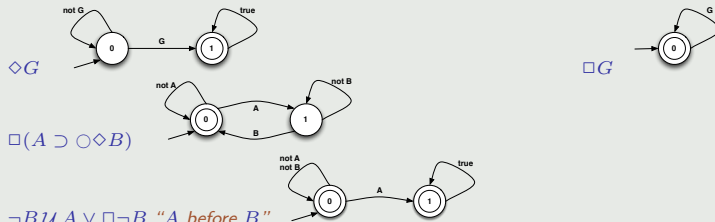
Key point

LTL_f formulas can be translated into a finite-state automaton on finite words $\mathcal{A}_\varphi$ such that:

$$t \models \varphi \text{ iff } t \in \mathcal{L}(\mathcal{A}_\varphi)$$

- in **linear time** if $\mathcal{A}_\varphi$ is an **Alternating Finite-state Automata (AFA)**;
- in **exponential time** if $\mathcal{A}_\varphi$ is an **Nondeterministic Finite-state Automaton (NFA)**;
- in **double exponential time** if $\mathcal{A}_\varphi$ is an **Deterministic Finite-state Automaton (DFA)**.

Example (Automata for some LTL_f formulas)



(online software for LTLf2DFA: <http://ltlf2dfa.diag.uniroma1.it>)

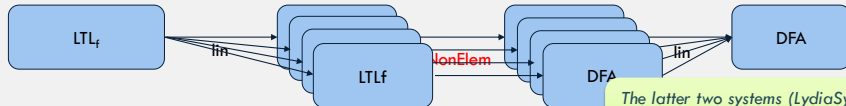
Monolithic tight bounds: [DeGiacomoVardi IJCAI2013/2015]



Monolithic via MONA [Zhu et al. IJCAI 2017]



Compositional [Bansal et al. AAAI2020], [DeGiacomoFavorito ICAPS2021], [Favorito Arxiv2023], [ZhuFavorito2025]



The latter two systems (LydiaSyft and Nike)
1st and 2nd places in 1st and 2nd edition
LTLf synthesis track at SYNTCOMP23/24

Better in practice!

LTL_f Satisfiability (φ SAT)

- 1: Given LTL_f formula φ
- 2: Compute AFA for φ (*linear*)
- 3: Compute corresponding NFA (*exponential*)
- 4: Check NFA for nonemptiness (*NLOGSPACE*)
- 5: Return result of check

LTL_f Validity (φ VAL)

- 1: Given LTL_f formula φ
- 2: Compute AFA for $\neg\varphi$ (*linear*)
- 3: Compute corresponding NFA (*exponential*)
- 4: Check NFA for nonemptiness (*NLOGSPACE*)
- 5: Return complemented result of check

LTL_f Logical Implication ($\Gamma \models \varphi$)

- 1: Given LTL_f formulas Γ and φ
- 2: Compute AFA for $\Gamma \wedge \neg\varphi$ (*linear*)
- 3: Compute corresponding NFA (*exponential*)
- 4: Check NFA for nonemptiness (*NLOGSPACE*)
- 5: Return complemented result of check

Thm: All the above reasoning tasks are PSPACE-complete. (Construction of NFA can be done while checking nonemptiness.)

As for the infinite traces.

LTL_f Satisfiability (φ SAT)

- 1: Given LTL_f formula φ
- 2: Compute AFA for φ (*linear*)
- 3: Compute corresponding NFA (*exponential*)
- 4: Check NFA for nonemptiness (*NLOGSPACE*)
- 5: Return result of check

LTL_f Validity (φ VAL)

- 1: Given LTL_f formula φ
- 2: Compute AFA for $\neg\varphi$ (*linear*)
- 3: Compute corresponding NFA (*exponential*)
- 4: Check NFA for nonemptiness (*NLOGSPACE*)
- 5: Return complemented result of check

LTL_f Logical Implication ($\Gamma \models \varphi$)

- 1: Given LTL_f formulas Γ and φ
- 2: Compute AFA for $\Gamma \wedge \neg\varphi$ (*linear*)
- 3: Compute corresponding NFA (*exponential*)
- 4: Check NFA for nonemptiness (*NLOGSPACE*)
- 5: Return complemented result of check

Thm: All the above reasoning tasks are PSPACE-complete. (Construction of NFA can be done while checking nonemptiness.)

As for the infinite traces.

LTL_f Satisfiability (φ SAT)

- 1: Given LTL_f formula φ
- 2: Compute AFA for φ (*linear*)
- 3: Compute corresponding NFA (*exponential*)
- 4: Check NFA for nonemptiness (*NLOGSPACE*)
- 5: Return result of check

LTL_f Validity (φ VAL)

- 1: Given LTL_f formula φ
- 2: Compute AFA for $\neg\varphi$ (*linear*)
- 3: Compute corresponding NFA (*exponential*)
- 4: Check NFA for nonemptiness (*NLOGSPACE*)
- 5: Return complemented result of check

LTL_f Logical Implication ($\Gamma \models \varphi$)

- 1: Given LTL_f formulas Γ and φ
- 2: Compute AFA for $\Gamma \wedge \neg\varphi$ (*linear*)
- 3: Compute corresponding NFA (*exponential*)
- 4: Check NFA for nonemptiness (*NLOGSPACE*)
- 5: Return complemented result of check

Thm: All the above reasoning tasks are PSPACE-complete. (Construction of NFA can be done while checking nonemptiness.)

As for the infinite traces.

LTL_f Satisfiability (φ SAT)

- 1: Given LTL_f formula φ
- 2: Compute AFA for φ (*linear*)
- 3: Compute corresponding NFA (*exponential*)
- 4: Check NFA for nonemptiness (*NLOGSPACE*)
- 5: Return result of check

LTL_f Validity (φ VAL)

- 1: Given LTL_f formula φ
- 2: Compute AFA for $\neg\varphi$ (*linear*)
- 3: Compute corresponding NFA (*exponential*)
- 4: Check NFA for nonemptiness (*NLOGSPACE*)
- 5: Return complemented result of check

LTL_f Logical Implication ($\Gamma \models \varphi$)

- 1: Given LTL_f formulas Γ and φ
- 2: Compute AFA for $\Gamma \wedge \neg\varphi$ (*linear*)
- 3: Compute corresponding NFA (*exponential*)
- 4: Check NFA for nonemptiness (*NLOGSPACE*)
- 5: Return complemented result of check

Thm: All the above reasoning tasks are PSPACE-complete. (Construction of NFA can be done while checking nonemptiness.)

As for the infinite traces.

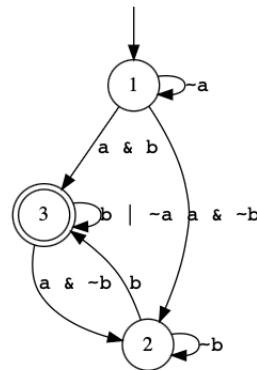
Example

Question: Check whether $\varphi_1 = \Box(a \supset \Diamond b) \wedge \Diamond a$ is satisfiable. Answer:

- Compute the automaton for φ_1 by using e.g., `ltl2dfa`
<http://ltlf2dfa.diag.uniroma1.it/> with input $G(a \rightarrow F b) \ \& \ F a$.
- Check if there is a path from the initial state to a final state, e.g., by least fixpoint, using existential preimage:
 $PreE(\mathcal{E}) = \{q \mid \exists a, s'. \delta(s, a, s') \text{ and } s' \in \mathcal{E}\}$

$$\begin{aligned} Win_0 &= \{3\} \\ Win_1 &= Win_0 \cup \{1, 2, 3\} \\ Win_2 &= Win_1 \cup \{1, 2, 3\} \\ &\text{Fixpoint!} \end{aligned}$$

- The initial state 1 is in *Win*, i.e., a final state is reachable from the initial one: the automaton accept at least a word and hence is **nonempty**.
- The formula φ_1 is **satisfiable**.



Example

Question: Check whether $\varphi_1 = \Box(a \supset \Diamond b) \wedge \Diamond a$ is satisfiable. Answer:

- Compute the automaton for φ_1 by using e.g., `ltl2dfa`
[ltl2dfa](http://ltlf2dfa.diag.uniroma1.it/) with input $G(a \rightarrow F b) \ \& \ F a$.
- Check if there is a path from the initial state to a final state, e.g., by least fixpoint, using existential preimage:

$$PreE(\mathcal{E}) = \{q \mid \exists a, s'. \delta(s, a, s') \text{ and } s' \in \mathcal{E}\}$$

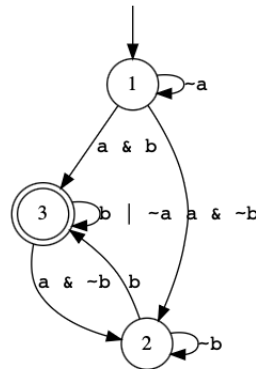
$$Win_0 = \{3\}$$

$$Win_1 = Win_0 \cup \{1, 2, 3\}$$

$$Win_2 = Win_1 \cup \{1, 2, 3\}$$

Fixpoint!

- The initial state 1 is in *Win*, i.e., a final state is reachable from the initial one: the automaton accept at least a word and hence is **nonempty**.
- The formula φ_1 is **satisfiable**.

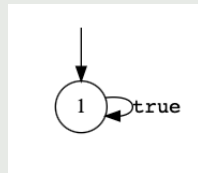


Example

Question: Check whether the logical implication $\Box(a \supset \Diamond b) \wedge \Diamond a \models \Diamond b$ holds. Answer:

- We solve it by satisfiability.
- $\Box(a \supset \Diamond b) \wedge \Diamond a \models \Diamond b$ holds iff $\varphi_2 = \Box(a \supset \Diamond b) \wedge \Diamond a \wedge \Box \neg b$ is unsatisfiable.
- Compute the automaton for φ_2 by using e.g., `ltl2dfa`
<<http://ltlf2dfa.diag.uniroma1.it/>> with input $G(a \rightarrow F b) \ \& \ F a \ \& \ G !b$.
- Check if there is a path from the initial state to a final state, e.g., by least fixpoint, using existential preimage:
 $PreE(\mathcal{E}) = \{q \mid \exists a, s'. \delta(s, a, s') \text{ and } s' \in \mathcal{E}\}$

$$\begin{aligned} Win_0 &= \{\} \\ Win_1 &= Win_0 \cup \{\} \\ \text{Fixpoint!} \end{aligned}$$



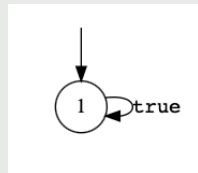
- The initial state 1 is not in Win , i.e., no final states are reachable from the initial one: the automaton does not accept any word, i.e., is **empty**.
- The formula φ_2 is **unsatisfiable**, and hence the **logical implication holds**.

Example

Question: Check whether the logical implication $\Box(a \supset \Diamond b) \wedge \Diamond a \models \Diamond b$ holds. Answer:

- We solve it by satisfiability.
- $\Box(a \supset \Diamond b) \wedge \Diamond a \models \Diamond b$ holds iff $\varphi_2 = \Box(a \supset \Diamond b) \wedge \Diamond a \wedge \Box \neg b$ is unsatisfiable.
- Compute the automaton for φ_2 by using e.g., `ltl2dfa`
[ltl2dfa](http://ltl2dfa.diag.uniroma1.it/) with input $G(a \rightarrow F b) \ \& \ F a \ \& \ G !b$.
- Check if there is a path from the initial state to a final state, e.g., by least fixpoint, using existential preimage:
 $PreE(\mathcal{E}) = \{q \mid \exists a, s'. \delta(s, a, s') \text{ and } s' \in \mathcal{E}\}$

$$\begin{aligned} Win_0 &= \{\} \\ Win_1 &= Win_0 \cup \{\} \\ \text{Fixpoint!} \end{aligned}$$



- The initial state 1 is not in Win , i.e., no final states are reachable from the initial one: the automaton does not accept any word, i.e., is **empty**.
- The formula φ_2 is **unsatisfiable**, and hence the **logical implication holds**.

Many Applications:

- **Reactive Synthesis**
- Planning for temporally extended goals
- MDP with non-Markovian goals
- Reinforcement Learning for non-Markovian tasks and safety constraints
- Declarative Process Specification in BPM
- ...

